# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1739**

---

COURSE OUTCOME COVERED

***CO1:*** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*( BL3)*

*Assessment Tool Weightage: 35%*

---

QUESTIONS: 1 **Duration : 1Hour**
**Total Marks:10**

**Annexure A**

**Constraint-Based Problem Solving**

---

***Code of Conduct:***

*I **B Reddy Balu (192312499)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

## 1. Problem Statement

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) subject to the following constraints:

C1. D1 cannot work the Night shift.

C2. D2 must work before D3 (Morning < Afternoon < Night).

C3. Only one doctor per shift.

C4. D3 cannot work the Morning shift.

C5. All doctors must be assigned exactly one shift.

**Task:** Apply Backtracking Search to find a valid assignment, showing all intermediate steps and constraint checks, and state the final valid schedule.

## 2. Problem Understanding

This is a **Constraint Satisfaction Problem (CSP)**. We must find a value for each variable (the doctors) from a given domain (the shifts) so that every constraint is satisfied at the same time.

There are 3 doctors and 3 shifts. Each doctor gets exactly one shift and no shift is shared, so a complete assignment is a one-to-one mapping (a permutation) of doctors to shifts. Without constraints there would be 3! = 6 possible schedules; the constraints reduce these to the valid ones.

### 2.1 Formal CSP Definition

**Variables (X):** X = { D1, D2, D3 }

**Domain (D):** For every variable the domain is { M, A, N }, where M = Morning, A = Afternoon, N = Night, with the order M < A < N.

**Constraints (C):**

| No. | Constraint | Type | Formal expression |
|---|---|---|---|
| C1 | D1 cannot work Night | Unary | D1 ≠ N |
| C2 | D2 works before D3 | Binary | order(D2) < order(D3) |
| C3 | Only one doctor per shift | All-different | D1 ≠ D2, D1 ≠ D3, D2 ≠ D3 |
| C4 | D3 cannot work Morning | Unary | D3 ≠ M |
| C5 | Every doctor has exactly one shift | Completeness | D1, D2, D3 each have exactly one value |

Shift order values used for C2: M = 1, A = 2, N = 3.

## 3. Constraint Analysis and Prioritisation

Before searching, the constraints are analysed to decide which are most useful for pruning.

**Priority 1: Unary constraints (C1, C4).** They involve one variable and shrink domains directly.

- C1: D1 ≠ N, so the domain of D1 becomes { M, A }.
- C4: D3 ≠ M, so the domain of D3 becomes { A, N }.

**Priority 2: Ordering constraint (C2).** D2 must work strictly before D3, so D2 cannot take Night (no shift comes after Night). D2 is therefore effectively restricted to { M, A }. If D3 = A, then D2 must be M.

**Priority 3: All-different constraint (C3).** Checked every time a value is assigned; it stops two doctors sharing a shift.

**Priority 4: Completeness (C5).** Checked at the end as the goal test.

**Reduced domains after analysis:**

| Variable | Original domain | After unary constraints and C2 reasoning |
|---|---|---|
| D1 | {M, A, N} | {M, A} |
| D2 | {M, A, N} | {M, A} |
| D3 | {M, A, N} | {A, N} |

Note: in the backtracking trace below, the **full original domains** {M, A, N} are used so that every constraint check is shown explicitly, as the question requires. The reduced domains are used afterwards to verify the answer.

## 4. Method: Backtracking Search

Backtracking is a form of **depth-first search** used to solve CSPs. It assigns one variable at a time and checks the constraints after every assignment. If a constraint is violated, it **backtracks** (undoes the last assignment) and tries the next value. If a variable has no legal value left, it backtracks to the previous variable.

### 4.1 Algorithm (Pseudocode)

```
function BACKTRACK(assignment, csp):
  if assignment is complete:
    return assignment                  # goal test (C5)
  var = SELECT-UNASSIGNED-VARIABLE(csp)    # order: D1, D2, D3
  for each value in DOMAIN(var):           # order: M, A, N
    if value is consistent with assignment (C1, C2, C3, C4):
      add {var = value} to assignment
      result = BACKTRACK(assignment, csp)
      if result is not failure:
        return result
      remove {var = value} from assignment   # backtrack
  return failure
```

### 4.2 Search Settings Used

- **Variable ordering:** D1, then D2, then D3.
- **Value ordering:** Morning (M), Afternoon (A), Night (N).
- **Constraint check:** after every assignment, all constraints involving only assigned variables are tested.

## 5. Step-by-Step Execution of Backtracking

### Step 1: Assign D1

Try **D1 = M (Morning)**.

- C1 (D1 ≠ N): M ≠ N → satisfied.
- C3 (one doctor per shift): no other doctor assigned → satisfied.

- C2, C4: do not involve D1 yet → not applicable.

**Result: Consistent.** Partial assignment: { D1 = M }. Move on to D2.

**Step 2: Assign D2 (with D1 = M)**

Try **D2 = M**.

- C3: Morning is already taken by D1 → VIOLATED.

**Result: Reject.** Try the next value.

Try **D2 = A (Afternoon)**.

- C3: Afternoon is free → satisfied.
- C1: does not involve D2 → not applicable.
- C2: D3 is not assigned yet, so it cannot be checked yet.

**Result: Consistent.** Partial assignment: { D1 = M, D2 = A }. Move on to D3.

**Step 3: Assign D3 (with D1 = M, D2 = A)**

Try **D3 = M**.

- C4 (D3 ≠ M): D3 = M → VIOLATED.
- C3: Morning is already taken by D1 → VIOLATED.

**Result: Reject.** Try the next value.

Try **D3 = A**.

- C3: Afternoon is already taken by D2 → VIOLATED.

**Result: Reject.** Try the next value.

Try **D3 = N (Night)**.

- C4 (D3 ≠ M): N ≠ M → satisfied.
- C3: Night is free → satisfied.
- C2 (D2 before D3): D2 = A (2) < D3 = N (3) → satisfied.
- C1 (D1 ≠ N): D1 = M → still satisfied.
- C5: D1, D2 and D3 each have exactly one shift → satisfied.

**Result: Consistent and complete.** The goal test passes, so the search returns this assignment.

### 5.1 Summary Trace Table

| Step | Variable | Value tried | C1 | C2 | C3 | C4 | Decision |
|---|---|---|---|---|---|---|---|
| 1 | D1 | M | ✓ | – | ✓ | – | Accept |
| 2 | D2 | M | – | – | ✗ (M taken) | – | Reject |
| 3 | D2 | A | – | pending | ✓ | – | Accept |
| 4 | D3 | M | ✓ | – | ✗ (M taken) | ✗ | Reject |
| 5 | D3 | A | ✓ | – | ✗ (A taken) | ✓ | Reject |
| 6 | D3 | N | ✓ | ✓ (A < N) | ✓ | ✓ | Accept: solution |

Legend: ✓ = satisfied, ✗ = violated, – = not applicable (variables involved not yet assigned).

### 5.2 Search Tree (Path to the First Solution)

```
ROOT { }
└── D1 = M  ✓
    ├── D2 = M  ✗ (C3)
    └── D2 = A  ✓
        ├── D3 = M  ✗ (C3, C4)
        ├── D3 = A  ✗ (C3)
        └── D3 = N  ✓  → GOAL: D1 = M, D2 = A, D3 = N
```

## 6. Final Valid Schedule

| Shift | Doctor assigned |
|---|---|
| Morning | D1 |
| Afternoon | D2 |
| Night | D3 |

**Final answer: D1 → Morning, D2 → Afternoon, D3 → Night.**

### 6.1 Verification Against All Constraints

| Constraint | Check with D1 = M, D2 = A, D3 = N | Result |
|---|---|---|
| C1: D1 cannot work Night | D1 works Morning | Satisfied |
| C2: D2 before D3 | D2 = Afternoon (2) < D3 = Night (3) | Satisfied |
| C3: One doctor per shift | M, A, N all different, each used once | Satisfied |
| C4: D3 cannot work Morning | D3 works Night | Satisfied |
| C5: Exactly one shift each | D1, D2, D3 each have one shift | Satisfied |

## 7. Exploring the Remaining Search Space (Completeness Check)

Backtracking is a complete search. If the search continues after the first solution (as if asking for all solutions), the rest of the tree is explored as follows.

**Branch D1 = M, D2 = N:**

- D3 = M: C3 violated (taken) and C4 violated → reject.
- D3 = A: C2 violated (D2 = N is after D3 = A) → reject.
- D3 = N: C3 violated (taken) → reject.
- Dead end, so backtrack.

**Branch D1 = A:**

- D2 = M: consistent.
- D3 = M: C3 and C4 violated → reject. D3 = A: C3 violated → reject.
- **D3 = N: C1, C2 (M < N), C3 and C4 all satisfied → second valid solution: D1 = A, D2 = M, D3 = N.**
- D2 = A: C3 violated (taken by D1) → reject.
- D2 = N: consistent with C3, but D3 has no legal value (M: C3, C4; A: C3; N: C3 violated) → dead end, backtrack.

**Branch D1 = N:** C1 is violated immediately → the whole subtree is pruned without further exploration.

### 7.1 Complete List of Valid Schedules

| Solution | Morning | Afternoon | Night |
|---|---|---|---|
| S1 (found first by the chosen ordering) | D1 | D2 | D3 |
| S2 | D2 | D1 | D3 |

The problem has exactly **two** valid schedules. Backtracking returns S1 because of the chosen variable and value ordering (D1 first, trying M before A before N).

### 7.2 Logical Check: Why Only These Two Work

- From C4, D3 ∈ {A, N}.
- If D3 = A, then D2 must be M (C2), which leaves D1 with N and violates C1. So D3 = A is impossible.
- Therefore D3 = N. D1 and D2 share M and A. Both orders satisfy C1 and C2, giving exactly S1 and S2.

This independent argument matches the result of the backtracking search.

## 8. Justification of Decisions

- **Why a CSP formulation?** The problem is defined by variables, finite domains and restrictions on combinations of values, which is exactly the structure of a CSP.
- **Why backtracking search?** It builds the solution incrementally and abandons a partial assignment as soon as a constraint fails, avoiding the $3^3 = 27$ raw combinations. For example, rejecting D2 = M after D1 = M removed that whole branch at once.
- **Why check constraints after every assignment?** Early detection of conflicts (e.g. C3 and C4 at D3 = M) prevents wasted work deeper in the tree.
- **Why this ordering?** Fixed orders D1, D2, D3 and M, A, N keep the trace simple and reproducible. Any other ordering would still find a valid solution because backtracking is complete.
- **Why prioritise unary constraints?** C1 and C4 restrict single variables and give the cheapest pruning. C2 and C3 then filter pairs of assignments.

- **Is the solution optimal?** The problem has no cost function, so any assignment satisfying all constraints is feasible and optimal. S1 and S2 are equally valid.

## 9. Conclusion

The hospital shift-assignment problem was modelled as a Constraint Satisfaction Problem with variables D1, D2, D3, domain {Morning, Afternoon, Night} and five constraints. Backtracking search, with constraint checks after each assignment, rejected the conflicting choices (D2 = M, D3 = M, D3 = A) and arrived at the valid schedule:

**Morning: D1, Afternoon: D2, Night: D3.**

A complete exploration of the search tree showed that the only other valid schedule is Morning: D2, Afternoon: D1, Night: D3. This confirms that the search is correct and that no other assignment satisfies all constraints.

## 10. References


1. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson, chapter on Constraint Satisfaction Problems.
2. Course notes, CSA17 Artificial Intelligence, SIMATS Engineering: Uninformed Search and Intelligent Agents.